Hardware Implementation of an Artificial Neural Network

Nazeih M. Botros, Ph.D., and M. Abdul-Aziz
Department of Electrical Engineering
Southern Illinois University
Carbondale, IL 62901.
Tel: 618-536-2364 Fax: 618-453-7455

Abstract-In this study we present a hardware implementation of a fully digital and fully interconnected feed forward back-propagation artificial network using Xilinx FPGAs. The network consists of an input layer with five nodes, a single hidden layer with four nodes, and an output layer with two nodes. These nodes are fully interconnected to each other between adjacent layers. Training is done off-line on a conventional digital computer where the final values of the weights are obtained. Each node is implemented with two XC3042 FPGAs and a 1K x 8 EPROM. The network is tested successfully by comparing the values of the output nodes for a different input patterns with those obtained from simulating the network on a PC. The number of FPGAs used can be significally decreased as well as the speed can be increased if 4K or higher family FPGA is used.

I. INTRODUCTION

In recent years, implementation of Field Programmable Gate Arrays (FPGA) in realizing complex hardware system has been accelerated. The relatively low cost and easiness of implementation and reprogramming of FPGA offer attractive features for the hardware designer in comparison with the VLSI technology [1]. Field programmable gate arrays are high density ASICS that can be configured by the user. They combine the flexibility of gate arrays with desktop programmability.

The artificial neural network implemented in this study is a three-layer back-propagation network. See Figure 1. The network is used to classify selected input patterns. It consists of 3 layers: an input layer with 5 nodes, a hidden layer with 4 nodes and an output layer with 2 nodes. The network is selected due to its high performance as a classifier and the easiness of its training procedure [2]. The input to the network is a continuous valued vector $x_1, x_2, \ldots x_5$. The output of the network is the class of the current input. The output of node j, y_j, is calculated as follows:

$$y_j = f\left[\left(\sum_i w_{ij}\, y_i\right) - \theta_j\right] \quad \text{.....(1)}$$

Where θ_j is the offset (bias) of node j, W_{ij} is the weight of the connection between node j and node i, and the function f is the sigmoid non-linearity:

$$f(\alpha) = \frac{1}{1+e^{-\alpha}} \quad \text{.....(2)}$$

Training of the network is carried out as follows:
i) Initialize the weights and offsets. Each node, except the input nodes, is assigned to an initial offset. The input nodes have no offset; they act as buffers.
ii) Starting from the output layer and going backward to the input layer, adjust the weights and offsets recursively until the weights stabilized. The weights and offsets are adjusted by using the formulas:

$$W_{ij}^{new} = W_{ij}^{old} + \mu\, \varepsilon_j\, x_i \quad \text{.....(3)}$$

$$\theta_j^{new} = \theta_j^{old} + \mu\, \varepsilon_j\, 1 \quad \text{.....(4)}$$

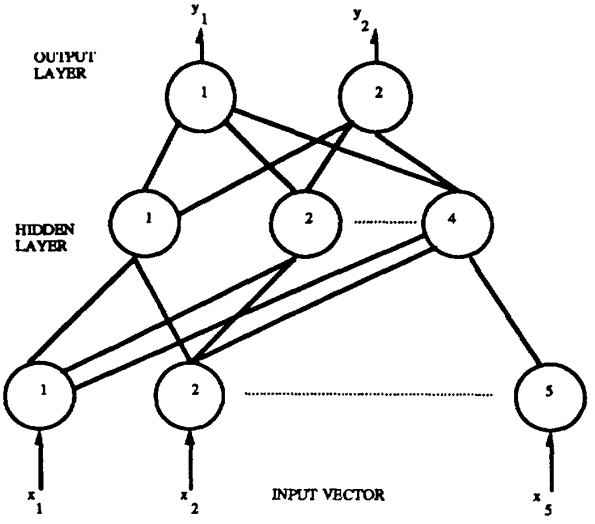

Figure 1. The Artificial Neural Network

Where μ is the gain factor and is assumed to be 0.5, ε_j is the error. In this study, the weights are considered stabilized if the value of each new weight is greater than 95% of its previous(old) value. If node j is an output node, then

$$\varepsilon_j = y_j\,(1-y_j)(d_j - y_j) \quad \text{.....(5)}$$

Where d_j is the desired output of node j and y_j is the actual output. The desired output for all output nodes is set to zero

0-7803-0999-5/93/$03.00 ©1993 IEEE

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on April 27,2025 at 10:11:07 UTC from IEEE Xplore. Restrictions apply.

except for the node that corresponds to the current input training set which is set to 1, [2]. If node j is a hidden node, then:

$$\varepsilon_j = y_j (1 - y_j) \sum_k \varepsilon_k W_{jk} \quad \text{.....(6)}$$

Where k is over all nodes in the layers above node j.

II. GENERAL ARCHITECTURE

Figure 2 shows the general architecture of the network. The input layer consists of five nodes (neurons), the hidden layer of four nodes and the output layer of two nodes. These nodes are fully interconnected to each other between adjacent layers. Training is done by off-line simulation of the network on a PC. The final values of weights are obtained at the end of training session. The input layer does no processing but simply buffers the data. The nodes in the other layers first form a weighted sum of their inputs. The 8-bit input to the hidden and output nodes y_i, is multiplied by 8-bit signed weights W_{ij} to form 16-bit signed product. The products (five in the hidden and four in the output layer) and a 16-bit signed bias value θ_j are accumulated into a 20-bit sum. The sum can be expressed as : $\left[\left(\sum_i w_{ij} y_i\right) - \theta_j\right]$.

The 20-bit sum in the accumulator is then scaled to a 10-bit value. The 10-bit scaling is selected because it is the minimum number of bits that can be retained without deteriorating the accuracy of the sum. This 10-bit scaled sum serves as the address of a 1K x 8 EPROM where a sigmoid activation function f is realized as a lookup table. The activation function produces an 8-bit output. Two's complement number system is employed to handle the multiplications and additions of negative numbers. This

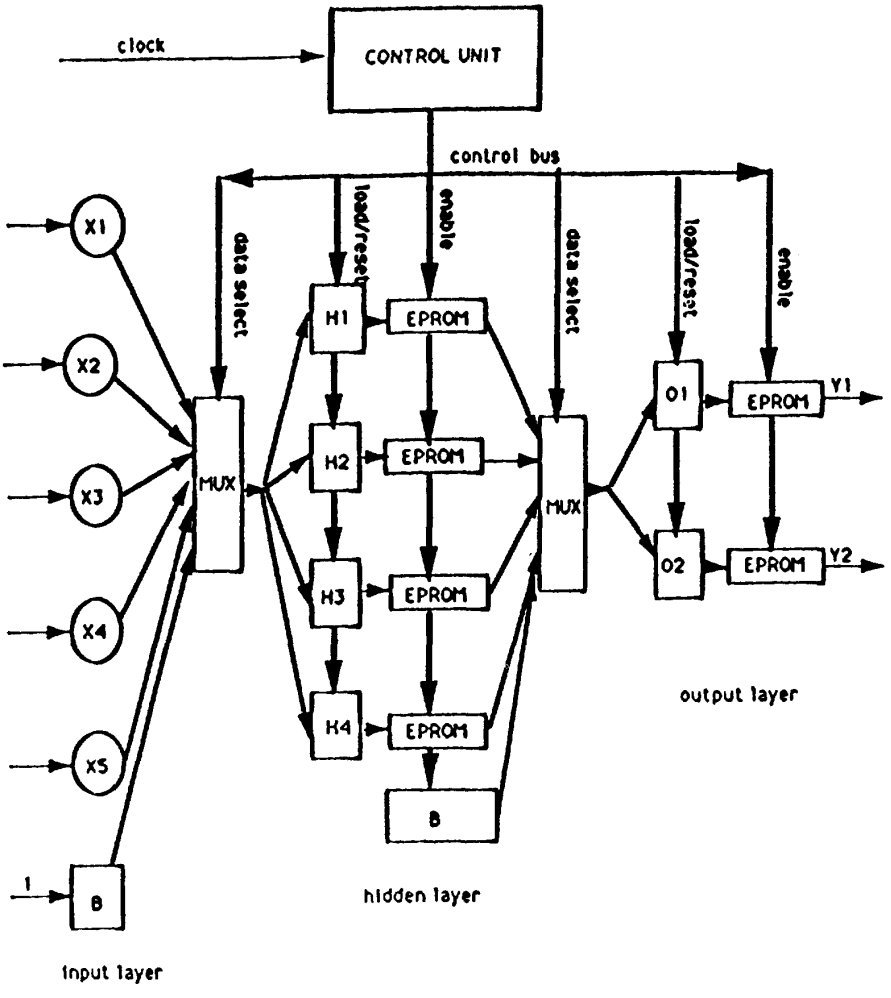

Figure 2. General Architecture of the Network.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on April 27,2025 at 10:11:07 UTC from IEEE Xplore. Restrictions apply.

output can be expressed as: $f\left[\left(\sum_i w_{ij} y_i\right) - \theta_j\right]$ where $f(\alpha) = \frac{1}{1+e^{-\alpha}}$. Figure 3 shows a schematic diagram of the processing flow for the hidden node. Each node in the network is built by two XC3042 FPGAs and a 1K by 8 EPROM. The first FPGA carries input latches and multipliers. The second carries a 20 bit fast adder/accumulator circuit and a scaling logic. These two FPGAs compute the weighted sum of five inputs (four in the output layer) and the bias value and then scale the result. The EPROM holds the activation functions for the node. The system clock is 4 MHz which is the maximum speed that could be achieved due to the use of slower EPROMs and two FPGAs per node. The entire network is driven by a micro programmed controller The controller generates a proper sequence of signals to control the timing for both layers. Computations for nodes of the same layer are done in parallel.

III. DETAILS OF THE ARCHITECTURE

Multiplication

Since the weights and biases are contants (pre-determined), multiplication of any number with them can be done in a look-up table fashion The CLBs (Configurable Logic Blocks) of the FPGA are programmed to realize these look-up tables. Multiplying an 8-bit number by an 8-bit constant produces a sixteen bit product. The 8x8 multiplication is broken into two-8x4 multiplications and one addition. The most significant partial product (8x4) is shifted by four bits before adding it to the least significant partial product. The shift is realized by physically shifting (routing) the most significant bits. One CLB is used to generate two bits of the product. The twelve bits of the partial product are generated by using six CLBs.

Summation

The next task performed by the nodes is to produce the sum of the partial products into a single 20-bit sum. The 20 bits are selected so that no overflow can happen. A 20-bit fast carry look-ahead adder is designed to carry out the summations. Each node in the hidden layer adds up ten 16-bit partial products and a bias into a 20-bit positive edge triggered accumulator to produce a single sum. The output nodes perform the same but for eight partial product.

Scaling and Activation Function

The final task performed by the nodes of hidden and output layers is the scaling and the application of activation (sigmoid) function. The final result of addition of all the partial products and the bias value is stored in a 20-bit accumulator. An investigation of the behavior of the sigmoid function shows that it saturates to approximately 1.0 when $\alpha \geq +7$ and saturates to approximately 0.0 if $\alpha \leq -8$. Accordingly the 20-bits is scaled to 9 bits.

Control Unit

A separate micro programmed controller drives the entire circuit. Two 4-bit asynchronous TTL counters are cascaded to generate the addresses for the control memory. The counters are driven by a 4 MHz system clock. The asynchronous clear inputs of the counters are connected to a push button switch. the ENP of the least significant counter is connected to the Q output of a JK flip flop whose Preset & Clear inputs are driven by two control signals produced by the control memory. See figure 4.

IV. RESULTS

The network is simulated by software and same input patterns are applied to both the software and the hardware network. In both cases the outputs are calculated. Table 1 shows the outputs and y_2 of both networks. See Figure 1. As shown in this table the hardware network (Hard W.) performs correctly. The hardware implementation computes 4 million interconnections (approx. 70,000 decisions) per second. This speed allows the implementation of the network in real-time applications.

V. DISCUSSION AND CONCLUSION

We have presented a successful hardware implementation of a simple artificial neural network. The implementation can be expanded to realize more complex networks. Reconfigurability and adaptability are the main features of the hardware. For a new application only the weights, biases and scaling parameters need to be re configured on the CLBs without changing the basic design. It is easily expandable just by adding more nodes with the same design.

Xilinx FPGAs and other alike FPGAs are found feasible and efficient tools for the design of neural nets. They offer acceptable densities without the cost and length design cycles of full custom circuits. Their reconfigurability and desktop programmability allow to make design changes at user's terminal, thereby avoiding the fabrication cycle times and non-recurring engineering charges. Although (due to our limited funds) the use of two XC3042 FPGAs (50 MHz) and a 1K x 8 EPROM (450ns) per node makes the network bulky, we found that its size and speed can be greatly improved by using higher density FPGAs. FPGA XC3090 can easily accommodate the circuits in the two XC3042 used in this study. It will also significantly reduce the size as well as increase the speed by eliminating the 55ns (approx.) delay between the I/O pins of two FPGAs. RAMs can be

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on April 27,2025 at 10:11:07 UTC from IEEE Xplore. Restrictions apply.

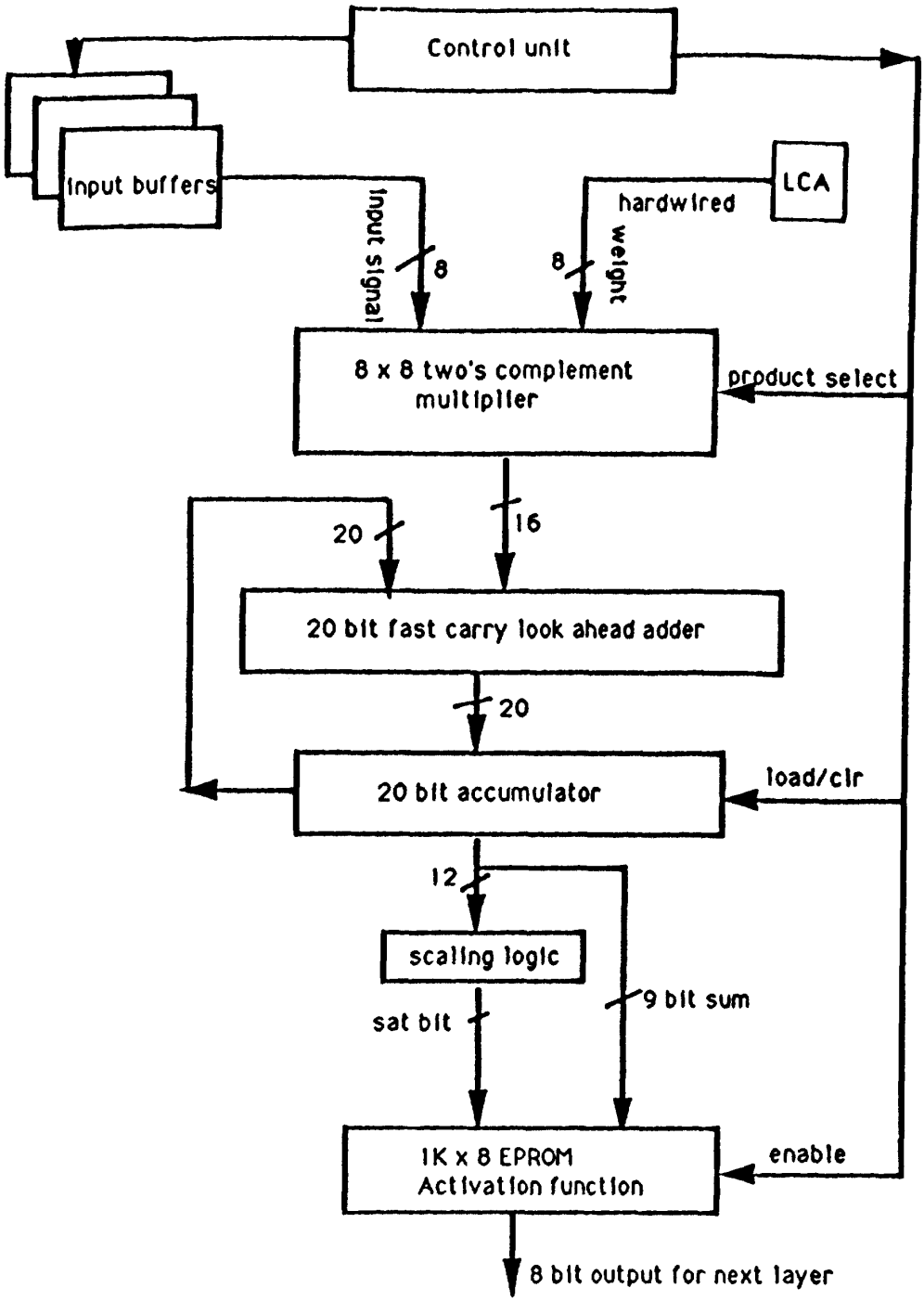

Figure 3.. Schematic of digital neuron (hidden unit)

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on April 27,2025 at 10:11:07 UTC from IEEE Xplore. Restrictions apply.

implemented inside the 45K FPGA series. These RAMs can be programmed for sigmoid lookup tables and can be downloaded with the bit stream during configuration, to further increase the speed and reduce the size to 1 chip per node. Use of pipeline techniques in each node as well as between successive layers of the network and higher speed EPROMs and FPGAs can also greatly increase the speed. Very high density FPGAs may provide room to build more than one neuron in one chip. At present time we are modifying the design to include on-chip training.

VI. REFERENCES

[1] C. E. Cox, and W. Ekkehard Blanz, " Ganglon- A Fast Hardware Implementation of a Connectionist Classifier, " IEEE-CICC Phoenix, Arizona, 1991.

[2] R. Lippmann, " An Introduction to Computing With Neural Nets," IEEE-ASSP, Magazine, 4-22, April 1987.

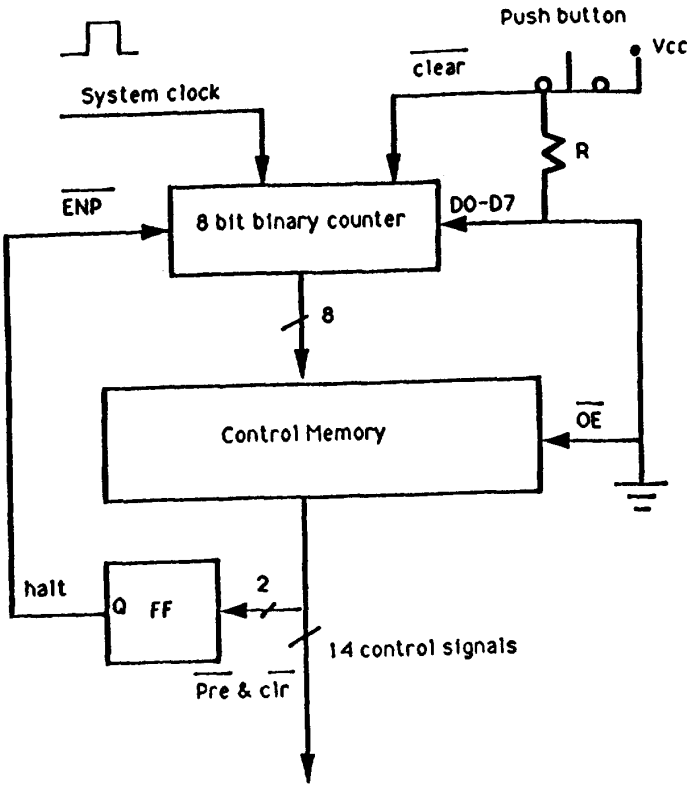

Figure 4. The Control Circuit

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on April 27,2025 at 10:11:07 UTC from IEEE Xplore. Restrictions apply.

Test Patterns					Output Y1		Output Y2	
					Soft W.	Hard W.	Soft W.	Hard W.
7	20	30	48	58	.98967	.99218	.00806	.00781
3	6	19	21	59	.99620	1	.01530	.01562
14	26	39	45	50	.98967	.99218	.00806	.00781
5	19	7	29	37	.98967	.99218	.00806	.00781
10	18	29	53	61	.98967	.99218	.00806	.00781
40	35	31	27	20	.03114	.03125	.95791	.96003
75	66	50	41	19	.03114	.03125	.95791	.06093
31	20	15	8	1	.03114	.03125	.95791	.96093
62	50	55	67	70	.74267	.74218	.20795	.20312
40	32	22	44	14	.03114	.03125	.95791	.96093
20	86	54	38	97	.98967	.99218	.00806	.00781
16	8	4	2	1	.03115	.03125	.95791	.96093
1	2	4	8	16	.99499	.99218	.01258	.01562
19	27	89	34	63	.98967	.99218	.00806	.00781
19	27	89	63	34	.56162	.51562	.41332	.40625
63	34	89	27	19	.03114	.03125	.95791	.96093
34	63	89	27	19	.03114	.03125	.05791	.96093
200	180	100	80	5	.03114	.03125	.95791	.06093
200	180	100	5	80	.03114	.03125	.95791	.96093
5	80	100	180	200	.98967	.99218	.00806	.00781
8	4	1	16	4	.60804	.60937	.42076	.40625

Table 1. Results of Software and Hardware Networks

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on April 27,2025 at 10:11:07 UTC from IEEE Xplore. Restrictions apply.